integer. The rest of the formatting expression is a literal match of all other characters, which have to match the input exactly, with the exception of spaces (including tabs): a single space in the formatting expression will match any amount of whitespace in the input.

Here's an example that uses reference variables as extra arguments (passed by reference) and a * modifier to suppress the assignment of the first conversion specification (the single "Gender:" character, which we're ignoring because who cares). Note that the input string `$str` has extra spaces and a tab character, which do not affect the outcome:

```
$str = "Gender:   M, Height:\t1.85m, Weight:   95kg";
sscanf($str, 'Gender: %*1s, Height: %fm, Weight: %dkg', $height, $weight);
var_dump($height, $weight);
```

```
Output:
float(1.85)
int(95)
```

`sscanf()` will also accept regular expressions as conversion specifications, though this is undocumented in the official PHP documentation:

```
$str = 'Gender: M, Height: 1.85m, Weight: 95kg';
var_dump(sscanf($str, 'Gender: %[^,], Height: %[0-9\.]m, Weight:
%[0-9]kg'));
```

```
Output:
array(3) {
  [0]=>
  string(1) "M"
  [1]=>
  string(4) "1.85"
  [2]=>
  string(2) "95"
}
```

The example above substitutes regular expression character classes instead of specifiers. `%[^,]` matches "any characters that are not commas," `%[0-9\.]` means "any decimal digits or dot characters" (the backslash is necessary because dots are